URL SAFETY CHECKER

Cyber Security Mini-Project Report

Prepared by: Sathish Kumar

Project domain: Cybersecurity / Python

1. Abstract

The URL Safety Checker is a beginner-level cybersecurity project developed using Python. It checks a user-entered URL for basic structural issues and common warning signs. The program identifies the hostname, checks HTTPS usage, and reports indicators such as an unusually long URL, multiple hyphens, an IP-address hostname, an @ symbol, or suspicious-looking words. The result is intended to improve awareness, not to certify a website as safe.

2. Problem statement

People may receive misleading links through messages, email, social media, or websites. A beginner may not know what warning signs to look for. This project provides a small, understandable tool that displays a few URL warning signs before the user decides what to do.

3. Objectives

• Understand the basic structure of a URL.

• Practise Python functions, strings, conditions, regular expressions, and URL parsing.

• Detect a small set of common warning indicators.

• Display the findings in a readable report.

• Learn the limitations of rule-based security checks.

4. Tools and technologies

Programming language: Python 3. Libraries: urllib.parse and re, both included with Python. External packages and a dataset are not required. The program can run in a terminal or a Python environment that supports input and output.

5. Methodology

The program accepts a URL from the user. It prepares the input for parsing, extracts the hostname, checks whether the hostname appears valid, and determines whether HTTPS is used. It then applies simple rules to identify warning signs. Finally, it prints the URL, format result, HTTPS result, hostname, and warnings.

6. System requirements

Software: Python 3 and a text editor or IDE. Hardware: any basic computer capable of running Python. Internet access is not required for the current version because the program does not contact the URL.

7. Implementation

The implementation is organized around a check_url() function and a main() function. check_url() returns the findings in a dictionary. main() obtains user input and prints the report. Standard-library modules are used to keep setup simple.

8. Testing

Test with a familiar example such as https://example.com, a URL without a scheme such as example.com, a long URL, and a URL containing unusual characters. Review the output and confirm that warnings match the rules implemented. Use harmless test strings only; do not visit suspicious websites.

9. Expected output

The terminal displays a heading, asks for a URL, and prints whether its format appears valid, whether it uses HTTPS, the hostname, and a numbered warning list. Output varies according to the entered URL.

10. Limitations

The checker uses basic rules and may produce false positives or miss dangerous links. HTTPS encrypts a connection but does not prove that a website is trustworthy. The project does not query threat-intelligence services, inspect redirects, scan webpage content, or detect malware.

11. Future enhancements

Possible improvements include checking redirects safely, integrating a reputable URL reputation API with appropriate permission and rate limits, adding a graphical interface, saving reports, and creating automated unit tests. API keys must be stored securely and never committed to a public repository.

12. Conclusion

This mini-project demonstrates how Python can be used to build a simple cybersecurity awareness tool. It helps the user notice selected URL warning signs and provides practice with parsing and conditional logic. The output must be treated as guidance only; users should still verify links independently.